

基于 DQN 的改进 NSGA-Ⅱ 求解多目标柔性作业车间调度问题^{*}

郑国梁¹, 张朝阳^{1,2}, 吉卫喜^{1,2}, 于俊杰¹

(1 江南大学机械工程学院, 无锡 214122;

2 江苏省食品制造装备重点实验室, 无锡 214122)

摘要:提出了一种基于深度 Q 网络 (Deep Q-Network, DQN) 改进的非支配排序遗传算法 (Non-dominated Sorting Genetic Algorithm Ⅱ, NSGA-Ⅱ), 以解决以最小化最大完工时间和最小化能源消耗为目标的多目标柔性作业车间调度问题 (Multi-Objective Flexible Job shop Scheduling Problem, MO-FJSP)。通过在 DQN 算法中定义马尔可夫决策过程和奖励函数, 考虑选定设备对完工时间和能源消耗的局部及全局影响, 提高了 NSGA-Ⅱ 初始种群的质量。改进的 NSGA-Ⅱ 通过精英保留策略确保运行过程中的种群多样性, 并保留了进化过程中优质的个体。将 DQN 算法生成的初始解与贪婪算法生成的初始解进行对比, 验证了 DQN 算法在生成初始解方面的有效性。此外, 将基于 DQN 算法的改进 NSGA-Ⅱ 与其他启发式算法在标准案例和仿真案例上进行对比, 证明了其在解决 MO-FJSP 方面的有效性。

关键词:深度 Q 网络算法; 多目标柔性作业车间调度问题; 奖励函数; 非支配排序遗传算法

中图分类号:TH165; TP18 **文献标志码:**A **文章编号:**1671-3133(2025)09-0001-11

DOI:10.16731/j.cnki.1671-3133.2025.09.001

Solving multi-objective flexible job shop scheduling problems with an improved NSGA-Ⅱ based on DQN

ZHENG Guoliang¹, ZHANG Chaoyang^{1,2}, JI Weixi^{1,2}, YU Junjie¹

(1 School of Mechanical Engineering, Jiangnan University, Wuxi 214122, China;

2 Jiangsu Key Laboratory of Advanced Food Manufacturing Equipment and Technology, Wuxi 214122, China)

Abstract: An improved Non-dominated Sorting Genetic Algorithm Ⅱ (NSGA-Ⅱ) based on the Deep Q-Network (DQN) is proposed to solve the Multi-Objective Flexible Job shop Scheduling Problem (MO-FJSP) with the goals of minimizing makespan and energy consumption. The Markov decision process and a reward function are defined in the DQN algorithm, considering the influence of selected machines on makespan and energy consumption both locally and globally. This approach enhances the quality of the initial population of the NSGA-Ⅱ. The elite retention strategy of the NSGA-Ⅱ is improved to ensure population diversity during execution and preserve high-quality individuals throughout the evolutionary process. The effectiveness of the DQN algorithm in generating initial solutions is validated by comparing its initial solutions with those generated by a greedy algorithm. Furthermore, the improved NSGA-Ⅱ based on the DQN algorithm is compared with other heuristic algorithms on standard and simulation cases, demonstrating its effectiveness in solving MO-FJSP.

Keywords: Deep Q-Network (DQN) algorithm; multi-objective flexible job shop scheduling problem; reward function; Non-dominated Sorting Genetic Algorithm Ⅱ (NSGA-Ⅱ)

0 引言

随着《中国制造 2025》的实施, 智能制造已经成为我国制造业发展的重要方向。柔性作业车间调度问

题 (Flexible Job shop Scheduling Problem, FJSP) 是作业车间调度问题 (Job shop Scheduling Problem, JSP) 的扩展, 于 1990 年由 BRUKER 等人^[1]首次提出。FJSP 在 JSP 的基础上, 增加了设备选择的维度, 使得加工工序

^{*} 国家自然科学基金青年科学基金项目 (51805213)

可以在不同的设备上,而不同设备的加工时间各异,这进一步增加了调度问题的复杂性。在企业生产中,合理的调度方案可以显著降低生产成本、提高生产效率,因此,柔性作业车间调度问题一直是学术界广泛研究的重点。

目前,国内外学者针对 FJSP 提出了多种求解方法,包括数学规划法、元启发式算法、规则调度算法和机器学习算法等。数学规划法作为一种精确求解算法,适用于小规模调度问题,但当问题规模扩大时,传统的精确算法难以在合理时间内找到高质量的解。相比之下,元启发式算法能够在相对较短的时间内找到较优解,其包括进化算法和群体智能算法进化算法,通过模拟生物进化的过程,利用选择、交叉和变异操作不断优化解集。其中,遗传算法 (Genetic Algorithm, GA)^[2] 是进化算法的一种典型代表,利用自然选择和遗传机制寻找最优解。群体智能算法^[3] 通过模拟自然界中的群体行为 (如鸟群、鱼群等) 来进行优化,粒子群优化 (Particle Swarm Optimization, PSO) 算法作为该类算法的代表之一,模拟鸟群觅食行为以优化问题解。这些算法在解决 FJSP 时表现出了较强的适应性和有效性,尤其在大规模调度问题中具有突出表现。

随着当前社会各界对绿色制造的重视,越来越多的车间研究项目将生产能耗作为优化目标。WANG 等人^[4] 采用结合 5 种启发式方法的新型混合初始化策略和考虑有限重启的新重启策略,来解决低碳多目标灵活车间调度问题。姜一啸等人^[5] 以设备能耗、刀具磨损和切削液消耗为碳排放来源,建立以最小化碳排放量、最小化最大完工时间等为目标低碳调度模型,提出了一种改进带精英策略的非支配遗传算法并进行求解。SENG 等人^[6] 同样考虑完工时间和能源消耗对车间调度优化的影响,提出了改进非支配排序遗传算法,通过拥挤度和非支配度的计算来优化种群的选择。MENG 等人^[7] 为解决具有可控处理时间的灵活作业车间调度问题,设计了一种具有减速、关闭/开启和延迟 3 种节能策略的节能解码,并提出了一种高效的多目标混合蛙跳算法进行求解。

深度 Q 网络 (Deep Q-Network, DQN) 算法是一种将深度学习与强化学习相结合的算法,广泛应用于各种需要决策优化的问题,由 MNIH 等人^[8] 首次提出。DQN 通过利用深度神经网络近似 Q 值函数,可以将状态值或状态-动作值作为神经网络的输入,并通过神经网络将其映射为具体的动作。HAN 等人^[9] 结合了深

度卷积神经网络和强化学习,解决了基于析取图调度的面向订单的制造问题,使用了具有优先重放功能的双深度 Q 网络,来获取每个决策步骤的最佳组合动作策略。LEI 等人^[10] 通过使用析取图来表示柔性作业车间调度状态,从而使用图神经网络来对状态进行分析,提出了多动作深度强化学习算法来对柔性作业车间的工件排序和机器选择进行决策。CHEN 等人^[11] 将 GA 与 Q 学习 (Q-learning) 相结合,提出了一种自学习遗传算法 (Self-Learning Genetic Algorithm, SLGA),由遗传算法作为基本优化方法,并在强化学习的基础上智能性地调整其关键参数。李健等人^[12] 提出一种协同智能体学习方法对柔性作业车间调度问题进行求解,采用 2 种不同的强化学习算法对工件和机器进行策略学习和更新。张凯等人^[13] 提出集成了 5 种深度 Q 网络优化的 D5QN 算法来解决柔性作业车间调度问题。SONG 等人^[14] 基于异构图神经网络架构来捕获工件与设备之间的复杂关系,使用深度强化学习来端对端学习高质量的优先级调度规则,以解决复杂的柔性作业车间调度问题。LI 等人^[15] 解决考虑模糊加工时间的多目标柔性作业车间调度问题,提出了具有 2 种强化学习的方法,通过 Q-learning 算法调整基于分解的多目标进化算法的邻域数量,再利用历史信息引导个体选择更适合的可变邻域搜索方式。

深度强化学习算法在解决柔性作业车间调度问题方面具有广泛应用前景。如何更好地将深度强化学习应用于 FJSP,是当前研究的一个重要方向。

本文以完工时间和能源消耗为调度目标,建立了一个多目标柔性作业车间调度模型 (Multi-Objective Flexible Job shop Scheduling Problem, MO-FJSP)。在此基础上,提出了一种基于 DQN 的改进非支配遗传算法 (Improved Non-dominated Sorting Genetic Algorithm II based on the Deep Q-Network, DQN-NSGA-II) 来解决 MO-FJSP。该算法通过定义 DQN 的状态值、动作空间,并设计独特的奖励函数,使智能体能够为工序选取合适的设备。DQN 算法与启发式 NSGA-II 的结合具有以下优点。

1) DQN 算法仅需生成适用于 NSGA-II 初始种群数量的解集,而无需大范围生成解集,从而降低了 DQN 算法的计算时间。

2) 提高了 NSGA-II 初始种群的质量,使其在迭代过程中能够更快地获取最优解。

为了适应 DQN 算法生成的初始解集,本文设计了精英保留策略,以保证算法运算过程中的种群多样

性,防止陷入局部最优。通过经典案例和一个车间实际案例,验证了 DQN-INSGA-II 在实际生产调度中的有效性。

1 问题描述与模型的建立

柔性作业车间调度问题可以定义如下:在订单中,有若干工件 $J_i (i=1,2,\dots,n)$ 需要加工,车间中有若干设备 $M_k (k=1,2,\dots,m)$ 可进行加工。模型的约束条件如下。

1) 每个工件的工序是预先确定的,且必须按照既定的工艺路线依次加工。除首道工序外,每道工序在进行加工时,必须等待前一道工序完成后方可进行下一道工序的加工。

2) 同一道工序在不同设备上的加工时间和能源消耗是不同的。

3) 在任意时刻,同一台设备只能加工一道工序,并且任意工序只能在一台设备上加工。

4) 不考虑工件在设备之间的运输时间、装夹时间以及换刀时间。

本文中对模型描述和建立模型中的符号与含义如表 1 所示。

表 1 模型符号与含义

符号	含义
i	工件索引
j	工序索引
k	设备索引
n	工件数量
m	设备数量
$O_{i,j}$	第 i 个工件的第 j 道工序
$P_{i,j}$	工序 $O_{i,j}$ 的开始加工时间
$S_{i,j}$	工序 $O_{i,j}$ 的加工完成时间
$P_{i,j,k}$	工序 $O_{i,j}$ 在设备 M_k 上的加工能耗
$S_{i,j,k}$	工序 $O_{i,j}$ 在设备 M_k 上的加工完成时间
$M_{i,j,k}$	若 $O_{i,j}$ 可在 M_k 上加工,该值为加工时间,否则取 0

本文解决多目标柔性车间调度,目标包括:最小化最大完工时间、最小化能源消耗,其具体含义如下。

1) 最小化最大完工时间的目标 F_T 是缩短所有工件加工完毕后所需的最长时间,即:

$$F_T = \min [\max (S_{i,j})] \quad (1)$$

2) 最小化能源消耗的目标 F_p 是最小化所有工序在相应设备上加工的能源消耗总和,即:

$$F_p = \min \sum_{k=1}^m (M_{i,j,k} \cdot P_{i,j,k}) \quad (2)$$

2 模型求解

2.1 DQN 算法求解算法初始解

DQN 算法的理论基础是马尔可夫决策过程 (Markov Decision Process, MDP),其目的是使智能体 (agent) 通过与环境的交互,学习并执行复杂的决策任务。强化学习算法通常由五元组 (S, A, P, R, γ) 表示。其中, S 表示智能体当前的状态; A 表示智能体在当前状态下所采取的动作; P 表示状态转移概率; R 表示在当前状态 S 下智能体采取动作 A 系统给予的奖励; γ 表示折扣因子。DQN 算法结合了深度学习与强化学习的优势,利用深度 Q 网络近似状态-动作价值函数,解决了连续值函数问题。在本文中, DQN 算法的主要目的是为一组随机调度顺序中的每一道工序分配合适的加工设备,以最小化最终的完工时间和能源消耗。通过设计智能体的环境、动作和奖励函数,智能体能够选择兼顾当前步骤和全局优化的加工设备。相比贪婪算法, DQN 算法在生成初始解时具有以下优势。

1) 贪婪算法通常只能优化单一目标,在处理多目标问题时表现不佳;而 DQN 算法可以在高维空间中进行选择,同时兼顾多个目标。

2) 贪婪算法只能在当前步骤中选择最优解,难以考虑全局优化;而 DQN 算法在设备选择时能够同时考虑局部和全局的优化。

2.1.1 智能体构建

1) 智能体环境。

在 DQN 算法中,工序码 (Operation Sequence, OS) 与机器码 (Machine Sequence, MS) 是环境中的 2 个基本变量,这 2 个变量共同构成了一组调度规则,染色体编码方式示例如图 1 所示。

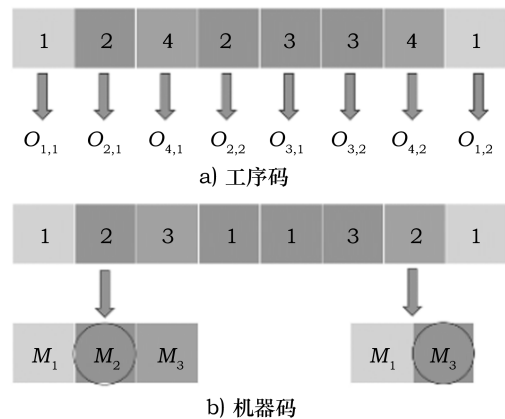


图 1 染色体编码方式示例

图 1a) 展示了一段工序码,其中包含 4 个工件,每

个工件对应 2 道工序。工序码中的数字代表工件编号,而该编号第几次出现则对应该工件的第几道工序。机器码是为每道工序选择加工设备的编码方式,机器码中的每个数字对应工序码中的相应位置。图 1b) 展示的是一段机器码,对应图 1a) 所示的工序码,机器码的每个数字对应工序码相应位置工序的设备。例如图 1b) 中第一个箭头对应工序 $O_{2,1}$ 的机器码,工序 $O_{2,1}$ 的可加工设备集合为 $\{M_1, M_2, M_3\}$, 机器码值为 2, 表示集合中的第二位设备 M_2 作为该工序的加工设备;同理,第二个箭头对应工序 $O_{4,2}$ 的机器码,按机器码值选择 M_3 作为该工序的加工设备。工序码与机器码的组合通过解码可得到该调度规则的最大完工时间和总能源消耗。

算法的每次迭代旨在生成一组机器码,即为每道工序选择加工设备。在每次迭代开始时,将环境定义为初始状态,工序码根据调度模型随机生成;初始状态下的机器码为空,其长度随着机器选择过程的进行而增加。机器选择过程按照工序码的顺序进行,当智能体为所有工序选择完对应的加工设备后,一次迭代结束。在每次算法迭代过程中,由于动作的随机性,会生成多组机器码,从而扩展智能体的探索范围,多组机器码的生成如图 2 所示。

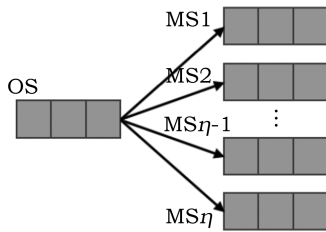


图 2 多组机器码的生成

2) 状态(State)描述。

状态值由已选择设备的工序和待选择工序共同决定。状态值分为两部分。

第一部分定义为在待选择工序的上一步为时间原点时刻的情况下,考虑所有设备的完工时刻。如果某个设备的完工时刻晚于原点时刻,则状态值设为该设备完工时刻与原点时刻之间的差值;如果完工时刻早于或等于原点时刻,则状态值设为 0。如果待选择的工序是工件的首道工序,则状态值直接设为 0。

第二部分定义为当前待选择工序在对应设备上加工,该工序所需的时间和能源消耗量拼接组成一维向量,若某一设备不可加工该道工序则将值设为-1。

状态值的最终定义为第一部分和第二部分的拼

接所组成的一维向量。

3) 动作(Action)设计。

在本文算法中,动作定义为智能体在某一状态下对下一道工序加工设备的选择。将状态值输入智能体后,智能体的输出是一个一维向量,其长度与设备数量相同,每个值代表选择对应设备的 Q 值。在实际应用中,选择具有最大 Q 值的设备。在网络训练过程中,首先提取能够加工该工序的设备的 Q 值,然后使用 SOFTMAX 函数将这些 Q 值转换为概率值。智能体在训练时通过概率选择动作,这样可以引入训练过程中的随机性,从而防止算法陷入局部最优解。

4) 奖励(Reward)定义。

合理的奖励设计是 DQN 算法的核心要素。奖励值是智能体在执行动作后由环境反馈的评价指标。奖励值由两部分组成,即时奖励(局部奖励)和全局奖励。即时奖励为针对智能体进行一次动作后给予的奖励,是比较当前状态下的所有可执行动作的优劣性。计算执行所有可执行动作后状态的最大完工时间、能源消耗,并分别计算与执行前的差值,将该差值定义为 $d_1^{q_1}$ 、 $d_2^{q_1}$,其中 q_1 表示可执行动作。定义最大完工时间奖励 $r_1^{q_1}$ 、能源消耗奖励 $r_2^{q_1}$,并初始化。对每个动作的局部奖励,定义如下:

$$r_{l_1}^{q_1} = \frac{1}{\left(\frac{1}{\min(d_{l_1})}\right)^{\varepsilon} \cdot (d_{l_1}^{q_1})^{\varepsilon}} \quad (3)$$

式中: l_1 取值为 $\{1, 2\}$, 分别表示最大完工时间和能源消耗; $\min(d_{l_1})$ 为所有动作奖励值的最小值; ε 为影响系数,一般取值为 2。

计算完成后采用 PCA 算法计算局部奖励。PCA 算法的主要作用是将多维的奖励通过数据投影处理为一维的局部奖励。将多维奖励值数据中心化,中心化函数 μ 为:

$$\mu = \frac{1}{m} \sum_{q_1=1}^m r^{q_1} \quad (4)$$

式中: m 为设备数量; r^{q_1} 为动作 q_1 的两组奖励值,因 r^{q_1} 表征不同机床的局部奖励值,故 $q_1 = 1, 2, \dots, m$ 。

将奖励值进行去中心化,即:

$$r_{\text{centered}}^{q_1} = r^{q_1} - \mu \quad (5)$$

式中: $r_{\text{centered}}^{q_1}$ 为去中心化的奖励值。

计算协方差矩阵,协方差矩阵的元素 Σ_{xy} 为:

$$\Sigma_{xy} = \frac{1}{m} \sum_{q_1=1}^m r_{\text{centered}}^{q_1}[x] \cdot r_{\text{centered}}^{q_1}[y] \quad (6)$$

式中: x 与 y 表示不同的动作; $r_{\text{centered}}^{q_1}[x]$ 为动作 x 下对

应设备索引的奖励值; $r_{\text{centered}}^{q_1}[y]$ 为动作 y 下对应设备索引的奖励值。

对协方差矩阵进行特征值分解,得到特征值 λ_1 、 λ_2 和对应特征向量 $\mathbf{v}_1$ 、 $\mathbf{v}_2$ 。两个特征值分别对应最大完工时间奖励和能源消耗奖励,其值大小,反映了相对应的奖励值所占局部奖励的比重,将原始数据投影至比重最大的主成分上来表示局部奖励值,即:

$$R_{\text{relevant}} = \mathbf{v}_k^T \cdot \mathbf{r}_{\text{centered}}^{q_1} \quad (7)$$

式中: $\mathbf{v}_k$ 为取最大特征值所对应的特征向量。

计算完成后得到的 R_{relevant} 是在该状态下所有可执行动作的奖励值,为了提高算法运算效率,每个状态下的奖励值均为所有动作的奖励值。局部奖励只能使智能体判断每个状态下可执行动作的优劣性,无法考虑该动作对全局的影响,因此引入全局奖励来让智能体获取每个动作对全局的影响。

在初始状态下,生成 φ 组相同工序码。在训练过程中,智能体通过 Q 值的大小随机选取动作,因此这 φ 组数据的机器码必不相同。在完成调度后,计算这 φ 组数据的最大完工时间 $\tilde{d}_1$ 、能源消耗 $\tilde{d}_2$,以及采用局部

奖励相同的方式计算初始奖励值 $\tilde{r}_{l_2}^{q_2}$, l_2 取值为 $\{1, 2\}$ 分别表示最大完工时间、能源消耗 (q_2 、 l_2 均为全局奖励参考,对应本文局部奖励 q_1 、 l_1)。则:

$$\tilde{r}_{l_2}^{q_2} = \frac{1}{\left(\frac{1}{\min(\tilde{d}_{l_2})} \right)^\varepsilon \cdot (\tilde{d}_{l_2}^{q_2})^\varepsilon} \quad (8)$$

计算完成后与计算局部奖励相同的方式利用 PCA 算法计算出 R_{global} 。 R_{global} 为一组相关联的动作赋值。每个动作的奖励 R 均分为局部奖励与全局奖励,定义:

$$R = \tau R_{\text{global}} + (1 - \tau) R_{\text{relevant}} \quad (9)$$

式中: τ 的值随训练迭代次数而增加,目的是为了在训练初期加大局部奖励的比例,而训练后期更加注重全局奖励。

2.1.2 DQN 算法求解算法初始解流程

DQN 算法通过环境与智能体的交互过程,使得智能体能够根据环境反馈来判断出合适的动作。DQN 算法模型训练流程如图 3 所示。图 3 中,Layer1 和 Layer2 分别为隐藏层层数和隐藏层单元格数。

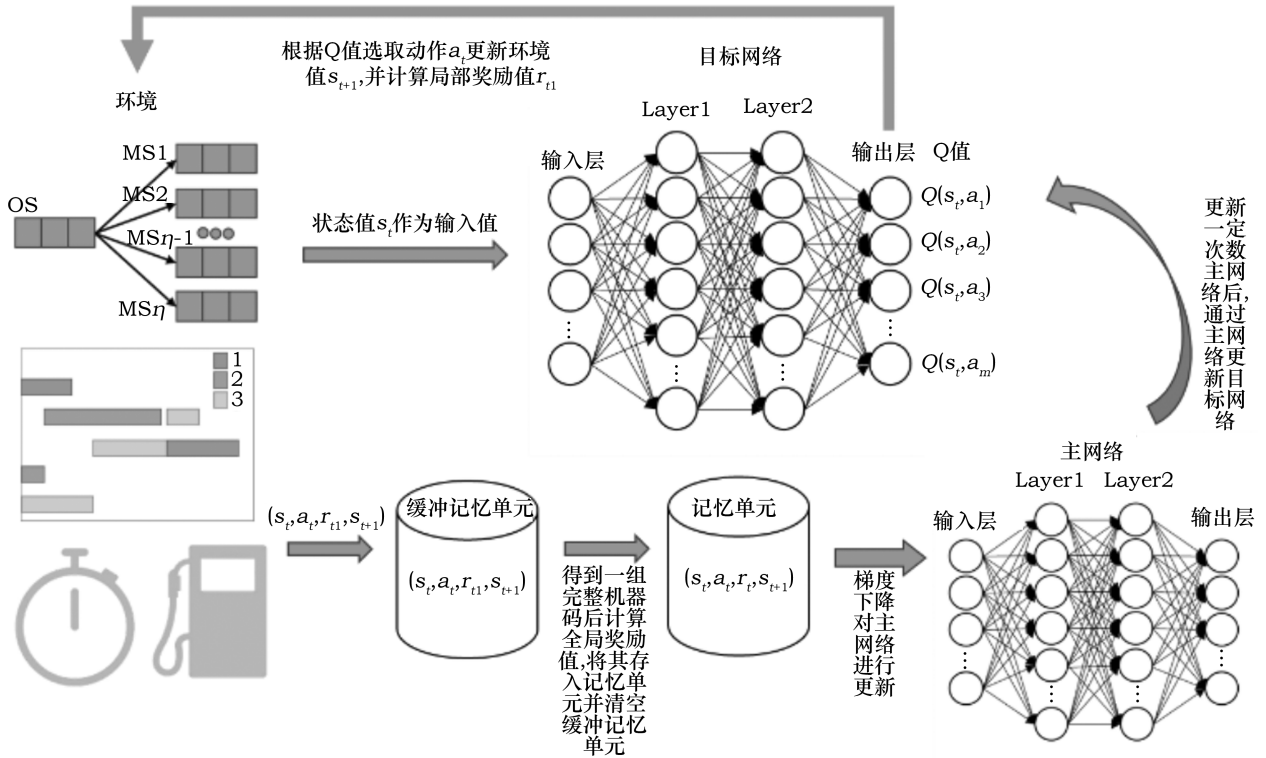


图 3 DQN 算法模型训练流程

主要训练步骤如下。

步骤 1: 初始化环境, 初始化环境的基本参数, 包括工件数量、工件工序数量、设备数量、工序可加工设

备集合、工序加工设备时间和能源消耗数据; 初始化 DQN 算法基本参数。

步骤 2: 训练过程, 随机生成一组用于训练的工序

码。从工序码的首位开始遍历,计算相应的状态值,并将其输入主网络以获得每个动作的 Q 值,确认所选动作并计算每个动作的局部奖励值,将其存入缓冲记忆单元。待遍历完所有工序码后,计算全局奖励,更新缓冲记忆单元的奖励值,并将遍历过程中的状态、动作和奖励值存入记忆元组,同时清空缓冲记忆元组。

步骤 3:目标网络训练,判断记忆元组中的存储数据是否达到 BATCH_SIZE 的规模,若达到指定规模则开始训练目标网络,采用均方误差 (Mean-Square Error, MSE) 作为损失函数,利用 Adam 动量优化算法来改进训练模型,每对目标网络训练一定次数后,利用目标网络更新主网络。

步骤 4:将步骤 2 与步骤 3 作为一组训练回合,重复步骤 2 与步骤 3 直至完成所有训练回合。训练结束后输出目标网络。

待训练完成后,利用所训练的模型生成一组用于遗传迭代的初始解。

2.2 非支配排序与选择操作

非支配排序 (Non-dominated Sorting, NS) 是一种在多目标优化问题中常用的排序方法,特别是在多目标进化算法中广泛应用。它用于将多目标优化问题中的解集根据支配关系进行分类,从而找到不同的 Pareto 前沿 (Pareto Front)。Pareto 前沿是多目标优化中的一个关键概念又称帕累托最优解集,它代表一组不可相互支配的解,即在这些解中,没有任何一个解在所有目标上都优于其他解。换句话说, Pareto 前沿上的每一个解在至少一个目标上是最优的,而在其他目标上也没有被其他解完全超越。非支配等级是指多目标优化问题中,对于给定的一组解,非支配等级用来衡量解的优劣。非支配等级的计算基于帕累托最优解的概念,即在一组解集中,一个解的非支配等级越低,表示该解越接近帕累托最优解集。

选择操作将采取二元锦标赛的方式进行。首先从初始种群中任意选择 2 个个体,比较 2 个个体的非支配等级,选择非支配等级高的个体;若两者的非支配等级相同,则任意选择一个。重复上述步骤,直至选择出与初始种群数量相等的个体,以便进行后续的交叉与变异操作。

2.3 交叉与变异操作

交叉与变异操作是促进种群进步的重要步骤。交叉操作是通过改变 2 个个体的染色体片段从而生成新的个体,变异操作通过随机改变种群个体的染色体

片段从而生成新的个体。这 2 个操作均可丰富种群的个体多样性。

交叉操作如图 4 所示,在父代种群中,通过随机选择 2 个个体进行交叉操作。具体而言,首先随机选取 ω 个工件号 (在本文示例中,选择了工件 1 和工件 2); 然后在父代个体中确定所选工件号的位置;接着分别交换 2 个父代个体在相应位置上的工序码和机器码;最后生成相应的子代染色体。

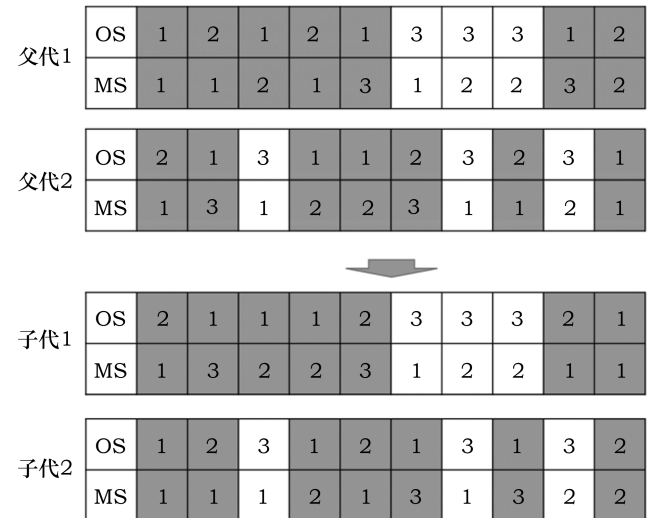


图 4 交叉操作

变异操作如图 5 所示,在父代个体中随机选择 σ 个工件号 (本文示例选择了工件 2 和工件 3),按顺序将其提取并进行重新排列。重新排列后的基因值将被填回染色体中。随后,随机选取基因位替换为不同于当前加工设备的另一台设备。

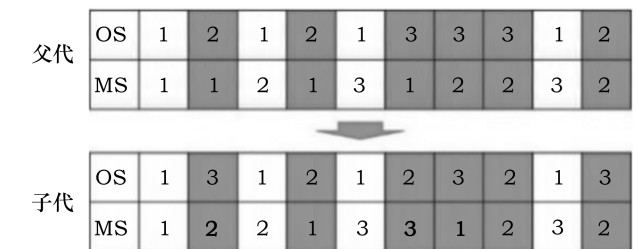


图 5 变异操作

2.4 精英保留策略

随着算法的迭代,大量的个体将会往第一非支配等级聚集,从而导致种群多样性降低,使得算法局部收敛,并导致优质个体被破坏。为了保证算法在迭代过程中的种群多样性并保证优质个体不被破坏,本文提出一种改进精英保留策略,同时兼顾了优质个体不被破坏和保证种群多样性。

父代种群数目为 N ,在完成交叉与变异操作后生成一组子代种群,数目为 N ,将父代种群和子代种群合并后,生成一组数目为 $2N$ 的合并种群。将合并种群进行非支配排序,计算合并种群各个非支配等级的种群数目 N_i ,合并种群在各个非支配等级选取的种群概率 p_i 为:

$$p_i = \sqrt{1 - \frac{N_i}{2N}} \left[(e^{-\frac{g}{d \cdot G}} - 1) + 1 \right] \quad (10)$$

式中: g 为当前迭代次数; d 为非支配等级; G 为算法迭代次数。

在算法前期,非支配等级低的个体被保留的概率较大,这是由于前期种群质量较低,应更大幅度地保留非支配等级低的个体。非支配等级低的个体被保留的概率随着算法的迭代过程而降低。具体步骤如下。

1) 将父代种群和子代种群合并为合并种群,并重新计算种群非支配等级。

2) 从合并种群中非支配等级为 1 的个体开始,通过对应非支配等级的选取概率判断是否保留该个体,当保留下来的个体数目为 N 时停止操作。

3) 若遍历完所有个体后,总的个体数目没有达到 N ,则从剩余个体中继续重复上述操作直至被保留的个体数目达到 N 。

4) 将新生成的种群输出至外部精英种群,重新进行非支配排序并保留第一非支配等级的个体。

2.5 算法流程

本文 DQN-INSGA-II 流程如图 6 所示。

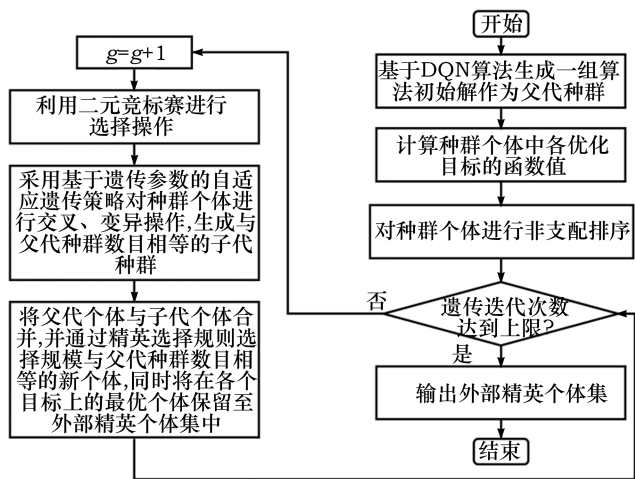


图 6 DQN-INSGA-II 流程

3 仿真实验

本文首先对 DQN 算法的主要超参数进行选择,通

过对比不同的超参数组合选择出合适的超参数进行训练网络;其次利用经典 BRdata 调度算例^[16]对 DQN 算法所生成的初始解与其他方法进行对比。最后将 DQN-INSGA-II 与传统 NSGA-II、帝国竞争算法 (Imperialist Competitive Algorithm, ICA)^[17]和引入参考点选择的非支配遗传算法 (NSGA-III)^[18]进行对比。算法运行环境为 Windows 10 64bit 下用 python3.7,深度学习基于 TensorFlow 2 上运行。

3.1 算法参数设置

在训练 DQN 算法时,选择合适的超参数对于提升算法性能至关重要。本文对 DQN 算法中的 3 个超参数进行对比选择,分别是学习率、隐藏层层数和隐藏层单元格数。学习率设置为 $[0.1, 0.01, 0.001]$,隐藏层层数设置为 $[2, 3, 4]$,隐藏层单元格数设置为 $[64, 128, 256]$,采用正交试验法对上述参数进行分析并选取最佳参数组合。为有效探索这些超参数的组合,采用 $L9(3^4)$ 正交表进行实验设计,包含 9 个实验组合。利用以上超参数进行训练模型,用超参数训练出的模型对算例 MK01 生成 100 个初始解,对初始解集的质量进行评估。评估采用 Spacing 指标^[19],该指标充分反映了非支配解集个体分布的均匀性,定义 Spacing 指标值 $Spacing$ 为:

$$Spacing = \sqrt{\frac{1}{N_o - 1} \sum_{a=1}^{N_o} (\bar{d} - d_a)^2} \quad (11)$$

式中: d_a 为个体 a 与其他个体之间距离的最小值; $\bar{d}$ 为个体 a 与其他个体之间距离的平均值; N_o 为解集的大小。

当 Spacing 指标值等于 0 时,代表着所有个体之间的距离都是等值分布,因此 Spacing 指标值越小,表示一组解集的质量越高。

DQN 算法的其他超参数保持一致,取初始探索率 $\varepsilon_1 = 1$,探索率折扣因子 $\Delta\varepsilon = 0.9995$,最小探索率 $\varepsilon_{\min} = 0.1$,DQN 算法迭代次数 $I_1 = 100$,初始局部奖励比例即折扣因子 $\gamma = 1$,局部奖励比例衰减率 $\Delta\gamma = 0.98$,最小局部奖励比例 $\gamma_{\min} = 0.75$,批次大小 $batch_size = 512$ 。正交矩阵及指标如表 2 所示。

对正交试验数据进行分析,绘制各因素均值效应图如图 7 所示。通过观察均值效应图,各因素水平在学习率为 0.01,隐藏层层数为 3 且单元格数为 256 时,Spacing 指标值最低。设置三者参数,重新训练模型,计算 Spacing 指标值并与上述 9 组数据对比,指标值对比如图 8 所示。从图 8 中可以看出,当因素水平为

[0.010, 3, 256] 时, Spacing 指标值最优, 因此训练模型时选择 [0.010, 3, 256] 作为训练参数。

表 2 正交矩阵及指标

实验组合	学习率	隐藏层层数	隐藏层单元格数	Spacing
1	0.001	2	64	0.045 4
2	0.010	2	128	0.042 2
3	0.100	2	256	0.043 9
4	0.010	3	64	0.038 8
5	0.100	3	128	0.041 8
6	0.001	3	256	0.038 9
7	0.100	4	64	0.042 3
8	0.001	4	128	0.039 5
9	0.010	4	256	0.040 1

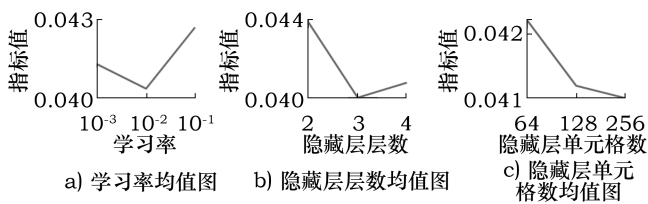


图 7 各因素均值效应图

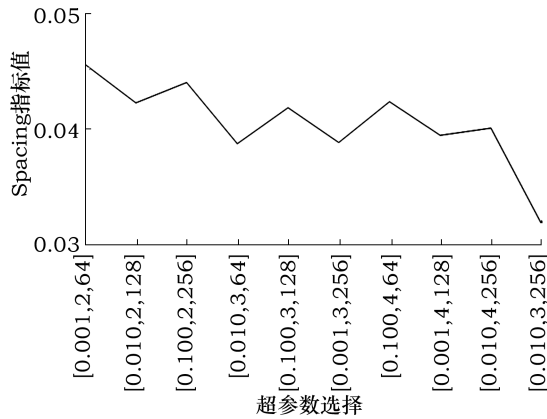


图 8 指标值对比

3.2 算法对比

为了验证 DQN 算法生成初始解的有效性, 分别对比 DQN 算法、普通贪婪算法和随机法对 10 个经典 BRdata 调度算例 (MK01 ~ MK10) 生成 100 个初始种群, 利用 Cov 指标^[20]对比 3 种算法所生成的初始解。Cov 指标反映了两个非支配解集之间的相互支配程度, 定义如下:

$$\text{Cov}(X, Y) = \frac{|b \in Y; \exists a \in X: a > b| + |a = b|}{|Y|} \quad (11)$$

式中: $a > b$ 为个体 a 支配个体 b , 表示个体 a 在每个目标中均优于个体 b ; $\text{Cov}(X, Y)$ 的含义是解集 Y 中被解集 X 中至少有一个支配或相同的比例, 因此若 $\text{Cov}(X, Y) = 1$, 则表示解集 Y 中所有的个体均可在解集 X 中

找到优于它的个体, 反之则表示解集 Y 中所有的个体在解集 X 中均没有优于它的个体。

$\text{Cov}(X, Y)$ 与 $\text{Cov}(Y, X)$ 在数学上没有必然的联系, 因此在进行算法指标分析时要同时考虑。

利用 DQN 算法、贪婪算法及随机法生成 100 组算法初始解, 计算 3 组算法所生成的初始解集之间的 Cov 指标, 算法生成初始解性能对比如表 3 所示, 表 3 中 Cov1 表示 DQN 算法与贪婪算法的 Cov 指标对比; Cov2 表示贪婪算法与随机法的 Cov 指标对比; Cov3 表示 DQN 算法与随机法的 Cov 指标对比。

表 3 算法生成初始解性能对比

标准算例	规模 $n \times m$	DQN 算法		贪婪算法		随机法	
		Cov3	Cov1	Cov1	Cov2	Cov2	Cov3
MK01	10×6	1.00	0.99	0.7	0.92	0.09	0.04
MK02	10×6	1.00	1.0	0.46	0.99	0.12	0.07
MK03	15×8	1.00	1.0	0.49	0.98	0.09	0.04
MK04	15×8	1.00	1.0	0.54	0.90	0.12	0.09
MK05	15×4	1.00	0.99	0.86	0.92	0.11	0.10
MK06	10×15	1.00	1.0	0.11	0.92	0.07	0.00
MK07	20×5	1.00	0.99	0.77	0.96	0.14	0.11
MK08	20×10	1.00	1.0	0.24	0.99	0.13	0.08
MK09	20×10	1.00	1.0	0.44	0.92	0.12	0.10
MK10	20×15	1.00	1.0	0.12	0.99	0.09	0.02

通过观察表中的数据可以发现 DQN 算法在生成初始解在各个案例均优于贪婪算法, 且遥遥领先随机法所生成的初始种群。并在算例 MK06 以及算例 MK10 中达到最优, 主要原因如下。

DQN 算法智能体在每一步的任务是为工序选择加工设备, 在选择设备时智能体包含两个方面的抉择, 其一是该选择对局部的影响, 其二是该选择对全局的影响, 若某工序可选设备较少, 则智能体的选择较少, 不同选择对全局的影响也较小, 而普通贪婪算法只考虑设备选择对局部的影响, 因此在设备数目较少的情况下 DQN 算法与普通贪婪算法的差值不大。而 DQN 算法在设备数目较多的情况下所生成的调度初始解效果要好于设备数目较少的情况。绘制 3 种算法生成算例 MK02、算例 MK06 及算例 MK10 散点图比较如图 9 所示。

为了进一步验证本文 DQN-NSGA-Ⅱ 的有效性, 将其与 GA、ICA、传统 NSGA-Ⅱ 和 NSGA-Ⅲ 进行对比。在相同条件下运行这些算法 10 次, 取遗传算法迭代次数 $I_2 = 100$, 传统 NSGA-Ⅱ 和 NSGA-Ⅲ 的初始解均采用随机生成, 取遗传交叉率 $\theta_1 = 0.5$, 遗传变异率 $\theta_2 = 0.1$ 。NSGA-Ⅲ 的参考点参考文献 [18] 方法生成。本文研

究的调度问题包含能源消耗指标,根据企业的实际生产数据将每台设备的加工能源消耗取值 $P_{i,j,k} \in$

$[0.5, 2]$ 。运行算法并记录各算法的最优结果,算法运行结果记录如表 4 所示。

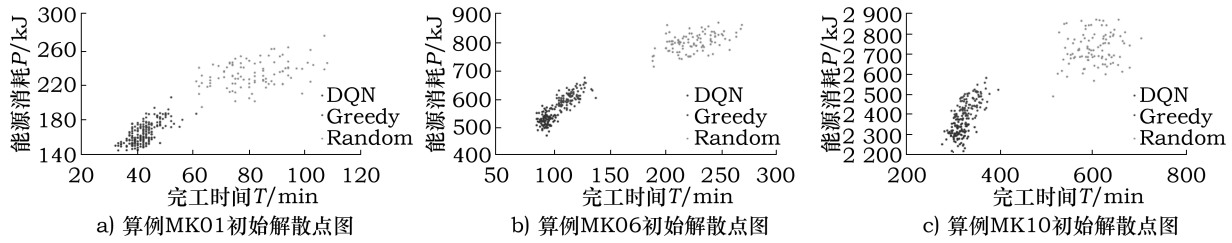


图 9 算例散点图比较

表 4 算法运行结果记录

算例	传统 NSGA-II		ICA		DQN-INSGA-II		NSGA-III	
	T/min	P/kJ	T/min	P/kJ	T/min	P/kJ	T/min	P/kJ
MK01	48	142	53	145	42	135.9	46	144.9
MK02	44	146.8	46	139.5	32	133.4	36	139.5
MK03	226	1012.9	230	867.5	213	814.4	208	861.2
MK04	85	193.2	86	208.7	72	199.9	76	193.2
MK05	199	552.4	203	566.7	193	559.9	193	552.4
MK06	120	572.4	120	594.2	84	445.5	96	523.5
MK07	199	621.4	218	632.1	160	597.6	199	618.4
MK08	558	2 089.2	558	2 111.3	527	1 980.7	542	2 046.4
MK09	425	2 554.3	448	2 567.9	383	2 386.1	405	2 507.2
MK10	321	2 277.4	367	2 409.2	276	2 186.7	297	2 203.9

绘制 4 种算法在算例 MK01、MK06 和 MK10 迭代过程中各目标值的收敛曲线和箱型图,分别如图 10~图 12 所示。

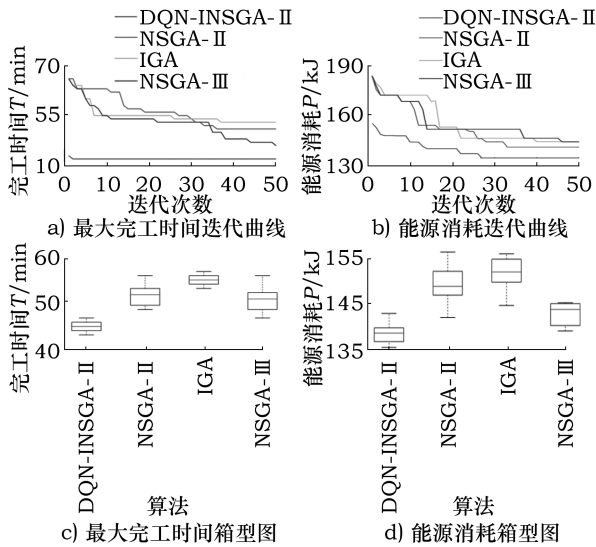


图 10 算例 MK01 迭代曲线和箱型图

通过观察这些曲线,可以发现 DQN-INSGA-II 的初始解与迭代结束的结果差异较小,其迭代曲线相比其他算法更为平缓,在算例 MK06 中,最大完工时间在整个迭代过程中没有发生变化;DQN-INSGA-II 的收敛

性相较于其他算法较为优秀。分析原因如下。

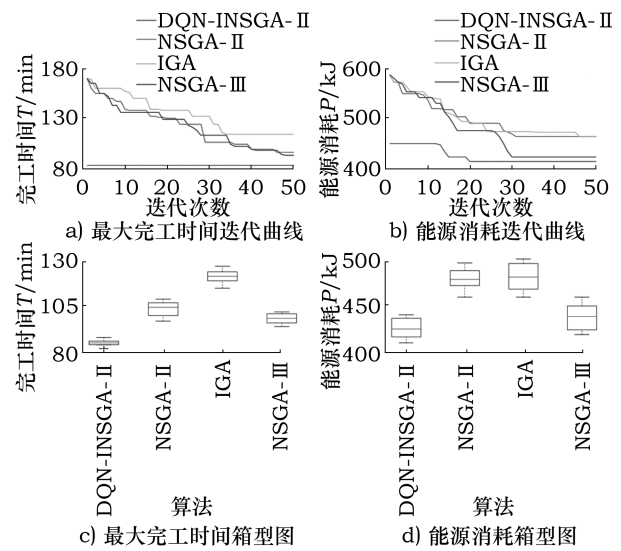


图 11 算例 MK06 迭代曲线和箱型图

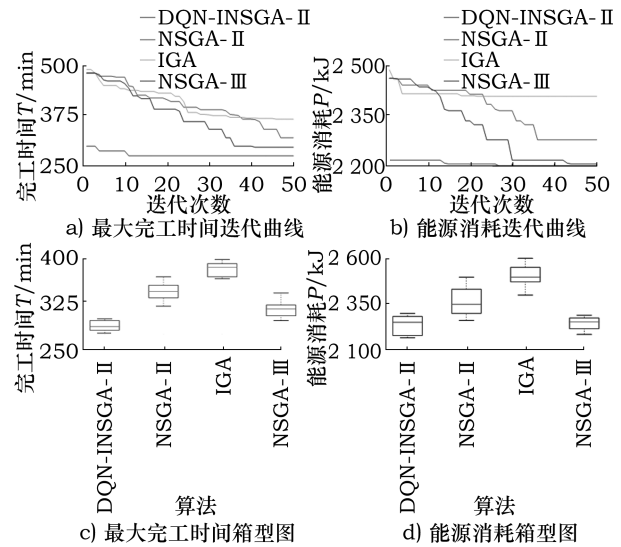


图 12 算例 MK10 迭代曲线和箱型图

1) DQN 算法生成初始解的方法在使得初始解的质量大大提升,甚至在生成初始解的过程中可以找到该算例下的最优解。由于有较高的初始解质量,算法

在迭代过程中所能进化的空间较小,因此迭代曲线相较于其他算法更加平缓。

2)DQN 算法生成初始解的方法使得算法初期的种群多样性减小,因此本文改进了 NSGA-Ⅱ 的精英保留策略,不仅可以保留优质个体在种群的延续,还能提高算法在迭代过程中的种群多样性,降低算法收敛于局部最优的可能性,使得算法的收敛性相较于其他算法更加优秀,因此 DQN-NSGA-Ⅱ 箱型图相较于其他算法的箱型图更加扁平。

表 5 离散车间生产调度仿真数据

工件号	工序号	可选设备集	加工时间/min	工件号	工序号	可选设备集	加工时间/min	工件号	工序号	可选设备集	加工时间/min
J ₁	O _{1,1}	1,2,3,4,5,7,8	5,3,5,3,3,10,9	J ₄	O _{4,1}	1,2,3,4,5,6,7,8	3,1,6,5,9,7,8,4	J ₇	O _{7,1}	1,2,3,4,5,7	5,4,2,6,7,10
	O _{1,2}	1,3,4,5,6,7,8	10,5,8,3,9,9,6		O _{4,2}	1,2,3,4,5,6,7,8	12,11,7,8,10,5,6,9		O _{7,2}	2,3,4,5,6,8	9,9,11,9,10,5
	O _{1,3}	2,4,5,6,7,8	10,5,6,2,4,5		O _{4,3}	1,2,3,4,5,6,7,8	4,6,2,10,3,9,5,7		O _{7,3}	2,3,4,5,6,8	8,9,3,8,6,10
J ₂	O _{2,1}	1,2,3,4,5,7	5,7,3,9,8,9	J ₅	O _{5,1}	1,2,3,4,5,7	3,6,7,8,9,10	J ₈	O _{8,1}	1,2,3,4,6,8	2,8,5,9,4,10
	O _{2,2}	2,3,4,5,6,7,8	8,5,2,6,7,10,9		O _{5,2}	1,3,4,5,6,7	10,7,4,9,8,6		O _{8,2}	1,2,3,4,5,7	7,4,7,8,9,10
	O _{2,3}	2,4,5,6,7,8	10,5,6,4,1,7		O _{5,3}	2,3,4,5,6,7	9,8,7,4,2,7		O _{8,3}	1,2,4,5,6,7,8	9,9,8,5,6,7,1
	O _{2,4}	1,2,3,4,6,8	1,4,5,6,10,7		O _{5,4}	1,2,4,5,6,7,8	11,9,6,7,5,3,6		O _{8,4}	1,3,4,5,6,7	9,3,7,1,5,8
J ₃	O _{3,1}	1,4,5,6,7,8	10,7,6,5,2,4	J ₆	O _{6,1}	1,2,3,4,5,6,8	6,7,1,4,6,9,10	J ₉	O _{9,1}	1,3,4,5,6,7	9,5,2,3,5,3
	O _{3,2}	2,3,4,5,6,7	10,6,4,8,9,10		O _{6,2}	1,3,4,5,6,7,8	11,9,9,9,7,6,4		O _{9,2}	2,3,5,6,7,8	8,5,6,2,4,3
	O _{3,3}	1,2,3,4,6,8	1,4,5,6,10,7		O _{6,3}	1,2,3,4,5,7	10,5,9,10,11,10		O _{9,3}	1,2,4,5,6,7	8,6,3,5,5,2

表 6 设备能源消耗

设备	加工能源消耗/(kJ·min ⁻¹)	设备	加工能源消耗/(kJ·min ⁻¹)
1	0.5	5	1.1
2	1.3	6	0.6
3	1.5	7	1.4
4	0.8	8	1.2

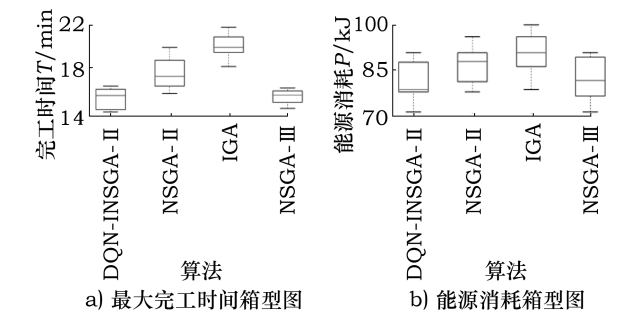


图 13 实例数据箱型图对比

从图 13 箱型图可以看出,在最大完工时间上,本文算法、NSGA-Ⅱ 及 NSGA-Ⅲ 三种算法均取得了最优结果,但本文算法的箱型图范围明显小于其他算法,这是由于初始解的优质性提高了算法在运行过程中的稳定性,使得算法在每次运行中均可以获取良好的解集;在能源消耗上,本文算法与 NSGA-Ⅲ 在求解的结果上相差不大,但箱型图中的中位线与上四等分线要优于 NSGA-Ⅲ。

3.3 仿真数据验证

本文的仿真数据来源于南通某电梯零/部件制造企业,以该企业某加工订单为案例,通过对该订单生产数据进行整理,构建为一个包含 9 个工件和 8 台设备的生产调度问题,离散车间生产调度仿真数据如表 5 所示,设备能源消耗如表 6 所示。需要指出的是,加工时间包括零件运输时间、装夹准备时间等因素。将 4 种算法各运算 8 次,绘制 8 次运算结果的箱型图,实例数据箱型图对比如图 13 所示。

图 14 所示为实例案例最小化最大完工时间甘特图。图 14 中,[0,2]为时间区间。

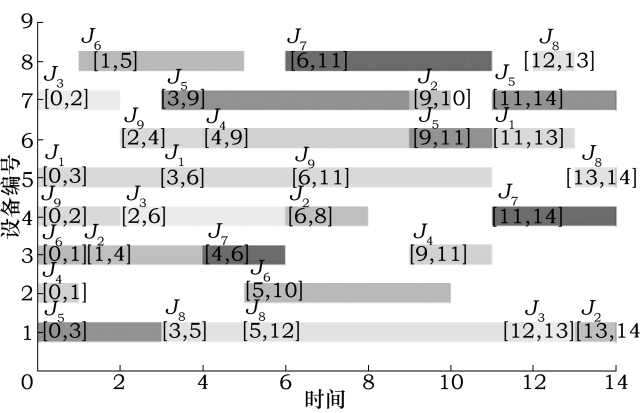


图 14 实例案例最小最大完工时间甘特图

4 结语

本文构建了一个多目标柔性作业车间调度模型,主要考虑最大完工时间和能源消耗,提出了一种基于 DQN 算法求解初始解的改进 NSGA-Ⅱ。结论如下。

1)针对多目标柔性作业车间的约束条件,设计了 DQN 算法的环境、动作和奖励函数,使得 DQN 算法智能体能够在给定一组工序顺序的情况下,为每道工序选择合适的加工设备。通过与贪婪算法的对比,验证了 DQN 算法在生成初始解方面的有效性。

2)将 DQN 算法生成的初始解应用于启发式算法 NSGA-II,同时改进了算法的选择、交叉与变异操作,确保算法运行过程中优质解集不被破坏,并设计了一种改进的精英保留策略以保证算法迭代过程中的种群多样性。通过在经典 BRdata 算例和仿真案例上与传统 NSGA-II、ICA 及 NSGA-III 进行对比,验证了本文算法在求解多目标问题上的有效性。

后续研究将集中于把 DQN 算法生成的模型应用于动态调度中。鉴于 DQN 算法能够快速为工序选择合适设备的特性,可将其应用于车间动态调度问题中。

参考文献:

- [1] BRUCKER P, SCHLIE R. Job-shop scheduling with multi-purpose machines[J]. Computing, 1990, 45(4): 369-375.
- [2] PEZZELLA F, MORGANTI G, CIASCETTI G. A genetic algorithm for the Flexible Job-shop Scheduling Problem[J]. Computers & Operations Research, 2008, 35(10): 3202-3212.
- [3] LIU Z, WANG J, ZHANG C, et al. A hybrid genetic-particle swarm algorithm based on multilevel neighbourhood structure for flexible job shop scheduling problem[J]. Computers & Operations Research, 2021, 135: 105431.
- [4] WANG Z, HE M, WU J, et al. An improved MOEA/D for low-carbon many-objective flexible job shop scheduling problem[J]. Computers & Industrial Engineering, 2024, 188: 109926.
- [5] 姜一啸, 吉卫喜, 何鑫, 等. 基于改进非支配排序遗传算法的多目标柔性作业车间低碳调度[J]. 中国机械工程, 2022, 33(21): 2564-2577.
- [6] SENG D, LI J, FANG X, et al. Low-Carbon Flexible Job-Shop Scheduling Based on Improved Nondominated Sorting Genetic Algorithm-II[J]. International Journal of Simulation Modelling, 2018, 17: 712-723.
- [7] MENG L, ZHANG C, ZHANG B, et al. MILP modeling and optimization of multi-objective flexible job shop scheduling problem with controllable processing times[J]. Swarm and Evolutionary Computation, 2023, 82: 101374.
- [8] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533.
- [9] HAN B A, YANG J J. Research on Adaptive Job Shop Scheduling Problems Based on Dueling Double DQN[J]. IEEE Access, 2020, 8: 186474-186495.
- [10] LEI K, GUO P, ZHAO W, et al. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem[J]. Expert Systems with Applications, 2022, 205: 117796.
- [11] CHEN R, YANG B, LI S, et al. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem[J]. Computers & Industrial Engineering, 2020, 149: 106778.
- [12] 李健, 李洹坤, 何鹏博, 等. 协同智能体强化学习算法的柔性作业车间调度方法研究[J]. 系统仿真学报, 2024, 36(11): 2699-2711.
- [13] 张凯, 毕利, 焦小刚. 集成强化学习算法的柔性作业车间调度问题研究[J]. 中国机械工程, 2023, 34(2): 201-207.
- [14] SONG W, CHEN X, LI Q, et al. Flexible Job-Shop Scheduling via Graph Neural Network and Deep Reinforcement Learning[J]. IEEE Transactions on Industrial Informatics, 2023, 19(2): 1600-1610.
- [15] LI R, GONG W, LU C. A reinforcement learning based RMOEA/D for bi-objective fuzzy flexible job shop scheduling[J]. Expert Systems with Applications, 2022, 203: 117380.
- [16] BRANDIMARTE P. Routing and scheduling in a flexible job shop by tabu search[J]. Annals of Operations Research, 1993, 41(3): 157-183.
- [17] 唐亮, 程峰, 吉卫喜, 等. 改进 ICA 求解柔性作业车间插单重调度问题[J]. 计算机工程与应用, 2023, 59(21): 303-311.
- [18] TIAN Y, XIANG X, ZHANG X, et al. Sampling Reference Points on the Pareto Fronts of Benchmark Multi-Objective Optimization Problems[M]. [S.l.]: [s.n.], 2018.
- [19] SCHOTT J R. Fault tolerant design using single and multicriteria genetic algorithm optimization[D]. Massachusetts: Massachusetts Institute of Technology, 1995.
- [20] ZITZLER E, THIELE L. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach[J]. IEEE Transactions on Evolutionary Computation, 1999, 3(4): 257-271.

作者简介:郑国梁,硕士研究生,主要研究方向为智能制造系统。

张朝阳,通信作者,副教授,博士,主要研究方向为智能制造系统与低碳制造。

E-mail:244435762@qq.com;cyzhang@jiangnan.edu.cn

收稿日期:2024-09-09